Security self-assessment

What can go wrong, whether we built the defence, where it lives, and what production would do instead. Permission tiers and keys are in `TECHNICAL.md`.

Risks

Risk	Built	Where in code	Production answer
Key custody	Partly. Five keys with separate roles, operator key behind a button step, model process holds none. Demo keys are generated at run time.	`agents.ts` sign(), `policy.ts` operator only rule, `Bond.topUp` operator check	Machine key in the secure element, operator key in a hardware device, lender key in the lessor's HSM, vendor key in our HSM. Rotation through `Bond.register` by the operator.
Oracle manipulation	Partly. No single source proves a field was sprayed, so three are layered: the farmer's confirmation (primary; the farmer pays, so has no reason to confirm work not done), the signed flight log with a geometry plausibility check (the faked log is held), and a 24 hour hold where the farmer can flag bad work for a human arbiter. Telemetry is simulated and signed by our keys.	`scenarios.ts` fleet run, tamper branch, `JobBoard.flag`	Secure element signs the track, farmer app signs the confirmation. A lie by one side fails one of three checks.
The pilot pays for the AI's mistakes	the bonded agent design (Part B, `DESIGN.md`), in this pass. Two bonds: the pilot bond is slashed only by work rules (`FalseConfirmation`, `OutsideGeofence`), the vendor bond only by signing rules (`PayeeNotAllowlisted`, `OverCap`).	`Bond.sol` rule table, `contracts.ts` mirror	Same. In production AI liability insurance sits above the vendor bond.
Evidence never leaves the pilot, so nothing is slashed	Part B, in this pass. Every intent hash is committed to `IntentLog` before the policy engine decides; the full intent is delivered to the farmer, the pilot, the lender and the vendor, blocked intents included. Only those parties can report on a live job.	`IntentLog.sol`, `Bond.slashByProof` "not in log" and "not a party" checks	Same. Delivery over the consortium chain's private channels.
The contract believes the reporter	Part B, in this pass. Every fact in a slash is read from the signed intent, `Bond`, `JobBoard` or `Lease`. The reporter only names the rule.	`Bond.slashByProof`, `JobBoard.isConfirmed`, `JobBoard.geofenceOf`	Same.

Risk	Built	Where in code	Production answer
A pilot drains the vendor bond with fake scams	Partly. Vendor penalties are a fixed amount with a daily cap; the bounty is half of a small penalty; hunters work only on shadow jobs.	`Bond.sol` vendor `penalty` and `dailyCap`	Instructions from the pilot's own channel count as the pilot's. Penalties go to the reporter as a capped bounty only.
Farmer and pilot collusion	Not built as code. Stated rule only.	`TECHNICAL.md` collusion answer	Jobs count toward the credit file only when the farmer is not the pilot's address, track and confirmation agree, and the fee came from an address with history.
Model outage	Built. The model only writes explanations; every decision is a tool. No key, timeout or error falls back to the template.	`agents.ts` explain(), `model.ts`	Same. The model is never on the path that moves money.
Chain reorg	Not built. `RealChain` waits for one receipt.	`chain.ts`	Consortium chain with finality (FISCO BCOS). On a public chain, wait for finality before releasing a fee.
Licensing	Not code.	`TECHNICAL.md` licensing answer	A licensed lessor in a sandbox is lender of record, Zishi services the contract, the wallet is a sub account of the pilot's business.
PIPL	Partly. Only an intent hash, a track hash and a confirmation go on chain; full intents stay with the parties of the job; no raw telemetry is stored.	`IntentLog.sol` (hash only), `scenarios.ts`	Consent in the job terms, data stays on the device and among the parties, hashes on chain.
Physical damage	Not built. Labelled out of scope. No insurance in the build.	Hold and flag in `JobBoard.flag`, arbiter named in the narration	The pilot's drone insurance plus a human dispute path. The bonds cover rule breaks only.
Crypto for mainland users	Not applicable to the demo. Testnet units are a settlement unit for evidence.	`chain.ts`	Bank money on a consortium chain. No public token.

Risk	Built	Where in code	Production answer
Judge gate false positives	Partly. A benign set runs with the attack set and must pass; the naive comparison shows what a regex only gate gets wrong. Pressure words pass with a note instead of a block. The classifier in `ml/gate.ts` can only add a hold, never a pass, and `selfcheck.ts` fails if any benign message is held.	`framing.ts` judge(), `attacks.ts` benign set, `ml/gate.ts` classify()	The same classifier behind the same pass, redact, hold, refuse interface, with the hold queue reviewed by a person.
Classifier drift	Partly. The classifier is trained at module load from a fixed corpus in `ml/corpus.ts`, so it cannot drift between requests. It can go stale: new attack phrasings it has never seen score low. The rules in `framing.ts` keep the last word on refuse, so a miss by the classifier is a miss only when the rules miss too.	`ml/gate.ts` classify(), threshold 0.8, `selfcheck.ts`	Retrain from the hold queue on a schedule, keep the held out split fixed across versions, and alert when f1 on it drops.
Synthetic training and eval data	Yes, and labelled. The classifier corpus is handwritten; the anomaly model's 2000 flights come from physical rules with noise and a fixed seed. Neither has seen a real field or a real attacker. The metrics are honest about the data they come from and the UI says held out and synthetic next to every number.	`ml/corpus.ts`, `ml/plausibility.ts` seed and tamper generator	Replace the flight set with signed logs from real drones and the corpus with real held messages, keeping the same metric code.
Metric gaming	Partly. `METRICS` in both models are computed at module load on a split chosen by a fixed seed; the agent has no write path to the corpus, the seed or the threshold. The 52 attacks and the benign set in `attacks.ts` are a second check the classifier cannot tune itself against. `/api/metrics` reads the module values; nothing is typed in.	`ml/gate.ts` METRICS, `ml/plausibility.ts` METRICS, `/api/metrics`	Corpus and flight set in a repository the agent runtime cannot push to, metrics recomputed in CI, and the held out split never used for tuning.
Ratchet overfits its own eval	Partly. The eval is one frozen week from a fixed seed, so the best strategy is best for that week. The search cannot cross the reserve rule (negative infinity), can move only four numbers, and every step is logged so a judge can see what changed. The strategy has no handle on the eval function.	`ratchet.ts` run(), `/api/ratchet`	Evaluate on several held out weeks and keep a change only if it improves on all of them. Changes to the strategy go through the operator key, not the ratchet.
Replay	Built. Spent set of intent hashes in the bond contract and its mirror; the attack set includes a replay case.	`Bond.sol` spent mapping, `attacks.ts` case 14	Same, plus nonces inside the typed intent.

Risk	Built	Where in code	Production answer
Operator key compromise	Partly. The operator key is the root of trust; only caps and top ups depend on it, and rate changes stay inside the signed band.	`Lease.setRate` band check, `Bond.topUp`	Hardware key that signs one displayed action at a time. Lender can freeze on a suspicious cap change.
Run API abuse	Partly. The per IP limiter in `ratelimit.ts` keeps its counters in one server instance's memory, so on a serverless host it is best effort and a determined caller can get past it. What holds: runs are bounded by `maxDuration`, the judge console caps the body at 64 KiB and never touches a live wallet, and the funded key is used only for requests carrying the demo token.	`web/src/app/api/run/route.ts`, `judge/route.ts`	Auth on the run endpoint, per key quotas, and a funded key that only a deployment secret holds.

Permission tiers

Four tiers, L0 read to L3 operator key, enforced in `PolicyEngine.evaluate()` on the signed intent and on chain by the contracts. Table, key table and what breaks when a key is lost or stolen are in `TECHNICAL.md`.

Trust boundaries

Boundary	What crosses it	Defence
Counterparty to agent	Messages from requesters, lenders, tool results	Escaped and framed as data, judge gate before the agent reads it
Model to engine	One sentence of explanation	Number grounding check, template fallback, no key in the model process
Agent to wallet	Typed intents	Policy engine, every rule as a line, allow or nothing moves
Engine to chain	Transactions	Same rules again in the contracts, every intent hash in the log first, slash by proof checked against records
Operator to engine	Cap and top up changes	Operator only rule, button step, band check
Learned components to engine	A risk score, an anomaly score, a plan, four strategy numbers	Scores can only add a hold. The plan is re-checked per job by `economics()`. The ratchet cannot break the reserve rule. The policy engine and contracts are unchanged.

Attack set

The set in `attacks.ts` runs at request time on the Evidence page and in `pnpm test`. Each case is an input to one layer (message, policy or contract) and must be blocked there. A benign set of ordinary requests must pass through the same gate. The naive comparison runs the same inputs through a regex only gate to show what it misses. Results are written by `pnpm evidence` to `evidence/`. See the live count on the Evidence page.

The hunter runs only against a copy of the agent on shadow jobs, which live in the same `JobBoard` under the same rules and move no lease money. Hunters never see a live job and cannot submit proof on one.

Their bounties come from the vendor bond under the same bounty rule as everyone else: half of the slash to the reporter, half to the treasury, each intent hash usable once.

Built, partly built, not built

Status	Item
Built	Policy engine with real signatures over typed intents
Built	Replay protection (spent set)
Built	Four tiers in code
Built	Judge gate with English and Chinese patterns
Built	Attack set with the naive comparison and the benign set
Built	Hunter bounty on a proven break, shadow jobs only
Built	Lender freeze on a slash or tampered data
Built	Button step for operator only actions
Built	Plan: economics check, reserve rule, decline
Built	Proof of solvency before a task
Built	Telemetry plausibility check, now a robust z-score model over four features with synthetic held out metrics
Built	Message classifier (logistic regression on hashed 3-grams) beside the rules in the gate, held out metrics
Built	Day planner as a bounded knapsack, replacing the greedy ranking
Built	Strategy ratchet against a frozen eval with the reserve rule as a hard constraint, logged per step
Built	Evidence and verify scripts
Built	Contracts written and tested in CI
In this pass (the bonded agent design (Part B, `DESIGN.md`))	Two bonds, the per-job intent log, proof checked against `JobBoard` records, party only reporting, hunters on shadow jobs, the starting floor, cover removed
Partly	Chain anchoring: self transfer with the event as calldata, needs a funded key
Partly	Model mode: needs a gateway key, falls back to templates
Not built	Contract deployment and ABI calls from the engine
Not built	Consent federation (accepted contacts, per contact budgets)
Not built	Ratchet against live data or several weeks (the built one runs on one frozen week)
Not built	Retraining the classifier or the anomaly model from real traffic
Not built	Seasonal minimum in `Lease` (decided, see `DESIGN.md`)

Logging

Every event the engine emits carries the actor, the intent, the policy lines with their tier, and the transaction hash. `pnpm evidence` writes the stream to `evidence/` and `pnpm verify` checks each hash against the explorer when the chain mode is real, or against the labelled simulated set when it is not.